E-Commerce Platform

Java Spring Boot Project - Complete Source Code and Documentation

Spring Boot 3.2 | Java 17 | JWT Security | JPA / H2 | REST API

1. Project Overview

Yeh ek complete online e-commerce backend hai jo Spring Boot par bana hai. Isme user registration/login (JWT), product catalogue, shopping cart, order checkout, mock payment aur admin management included hai.

Features

• Register / Login with JWT token aur BCrypt password hashing

• Role based access: USER aur ADMIN

• Products: search, pagination, admin CRUD

• Cart: add, update, remove with stock validation

• Orders: checkout (stock deduct), pay (mock), cancel (stock restore), order history

• Admin: saare orders dekhna aur status update (SHIPPED, DELIVERED)

• Global exception handling aur request validation

• Startup par sample products aur admin user auto-create

2. Tech Stack

Layer	Technology
Language	Java 17
Framework	Spring Boot 3.2.5
Security	Spring Security + JWT (jjwt 0.12.5)
Database	H2 in-memory (MySQL/PostgreSQL ready)
ORM	Spring Data JPA / Hibernate
Build	Maven

3. Folder Structure

```
ecommerce/
  README.md
  pom.xml
  src/
    main/
      java/
        com/
          shop/
            ShopApplication.java
            config/
              DataSeeder.java
            controller/
              AuthController.java
              CartController.java
              OrderController.java
              ProductController.java
            dto/
              Dtos.java
            exception/
              ApiException.java
              GlobalExceptionHandler.java
            model/
              CartItem.java
              Order.java
              OrderItem.java
              OrderStatus.java
              Product.java
              Role.java
              User.java
            repository/
              CartItemRepository.java
              OrderRepository.java
```

```
        ProductRepository.java
        UserRepository.java
      security/
        JwtFilter.java
        JwtService.java
        SecurityConfig.java
      service/
        CartService.java
        OrderService.java
resources/
  application.properties
```

4. How to Run

• Requirements: Java 17+ aur Maven 3.8+

• Project folder me jao aur chalao: mvn spring-boot:run

• Server start hoga: http://localhost:8080

• Default admin login: admin@shop.com / admin123

• Database in-memory H2 hai, restart par data reset hota hai

5. REST API Endpoints

Method	URL	Access
POST	/api/auth/register, /api/auth/login	Public
GET	/api/products?q=&page=0&size=10	Public
GET	/api/products/{id}	Public
POST/PUT/DELETE	/api/products[/{id}]	ADMIN
GET/POST	/api/cart	User
PUT/DELETE	/api/cart/{itemId}	User
POST	/api/orders (checkout)	User
GET	/api/orders, /api/orders/{id}	User
POST	/api/orders/{id}/pay, /cancel	User
GET	/api/admin/orders	ADMIN
PUT	/api/admin/orders/{id}/status?status=SHIPPED	ADMIN

6. Quick Test (curl)

```
TOKEN=$(curl -s -X POST localhost:8080/api/auth/register -H 'Content-Type: application/json' \
  -d '{"name":"Ravi","email":"ravi@test.com","password":"secret123"}' \
  | sed 's/.*"token":"\([^"]*\)".*/\1/')

curl -X POST localhost:8080/api/cart -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"productId":1,"quantity":2}'

curl -X POST localhost:8080/api/orders -H "Authorization: Bearer $TOKEN"
```

7. Production Notes

• app.jwt.secret ko environment variable se set karo

• H2 ki jagah MySQL/PostgreSQL use karo

• pay() me real gateway (Razorpay/Stripe) integrate karo

• Concurrent orders ke liye product par optimistic locking add karo

8. Source Code

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
  </parent>
  <groupId>com.shop</groupId>
  <artifactId>ecommerce</artifactId>
  <version>1.0.0</version>
  <properties><java.version>17</java.version></properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
        web</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
        jpa</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
        security</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
        validation</artifactId></dependency>
    <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><scope>runtime</scope></dependency>
<dependency><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId><optional>true</optional></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-
        api</artifactId><version>0.12.5</version></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-
        impl</artifactId><version>0.12.5</version><scope>runtime</scope></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-
        jackson</artifactId><version>0.12.5</version><scope>runtime</scope></dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration>
          <excludes><exclude><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId></exclude></excludes>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

src/main/resources/application.properties

```
spring.datasource.url=jdbc:h2:mem:shopdb
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
app.jwt.secret=change-this-secret-key-to-at-least-32-characters-long-please
app.jwt.expiration-ms=86400000
```

src/main/java/com/shop/ShopApplication.java

```
package com.shop;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ShopApplication {
    public static void main(String[] args) {
        SpringApplication.run(ShopApplication.class, args);
    }
}
```

src/main/java/com/shop/config/DataSeeder.java

```
package com.shop.config;

import com.shop.model.Product;
import com.shop.model.Role;
import com.shop.model.User;
import com.shop.repository.ProductRepository;
import com.shop.repository.UserRepository;
```

```
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;
import java.math.BigDecimal;

@Component
@RequiredArgsConstructor
public class DataSeeder implements CommandLineRunner {
    private final UserRepository users;
    private final ProductRepository products;
    private final PasswordEncoder encoder;

    @Override
    public void run(String... args) {
        if (users.count() == 0) {
            User admin = new User();
            admin.setName("Admin");
            admin.setEmail("admin@shop.com");
            admin.setPassword(encoder.encode("admin123"));
            admin.setRole(Role.ADMIN);
            users.save(admin);
        }
        if (products.count() == 0) {
            add("iPhone Case", "Shockproof back cover", "499.00", 100, "Accessories");
            add("Wireless Mouse", "2.4GHz silent mouse", "799.00", 50, "Electronics");
            add("Running Shoes", "Lightweight daily trainers", "2499.00", 30, "Fashion");
            add("Java Programming Book", "Learn Java step by step", "649.00", 40, "Books");
            add("Bluetooth Speaker", "Portable 10W speaker", "1599.00", 25, "Electronics");
        }
    }

    private void add(String name, String desc, String price, int stock, String cat) {
        Product p = new Product();
        p.setName(name);
        p.setDescription(desc);
        p.setPrice(new BigDecimal(price));
        p.setStock(stock);
        p.setCategory(cat);
        products.save(p);
    }
}
```

src/main/java/com/shop/controller/AuthController.java

```
package com.shop.controller;

import com.shop.dto.Dtos.*;
import com.shop.exception.ApiException;
import com.shop.model.Role;
import com.shop.model.User;
import com.shop.repository.UserRepository;
import com.shop.security.JwtService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final UserRepository users;
    private final PasswordEncoder encoder;
    private final JwtService jwt;

    @PostMapping("/register")
    public AuthResponse register(@Valid @RequestBody RegisterRequest r) {
        if (users.existsByEmail(r.email()))
            throw new ApiException(HttpStatus.BAD_REQUEST, "Email already registered");
        User u = new User();
        u.setName(r.name());
        u.setEmail(r.email());
        u.setPassword(encoder.encode(r.password()));
        u.setRole(Role.USER);
        users.save(u);
        return new AuthResponse(jwt.generate(u.getEmail(), u.getRole().name()), u.getEmail(), u.getRole().name());
    }
    @PostMapping("/login")
```

```
    public AuthResponse login(@Valid @RequestBody LoginRequest r) {
        User u = users.findByEmail(r.email())
                .filter(x -> encoder.matches(r.password(), x.getPassword()))
                .orElseThrow(() -> new ApiException(HttpStatus.UNAUTHORIZED, "Invalid email or password"));
        return new AuthResponse(jwt.generate(u.getEmail(), u.getRole().name()), u.getEmail(), u.getRole().name());
    }
}
```

src/main/java/com/shop/controller/CartController.java

```
package com.shop.controller;

import com.shop.dto.Dtos.CartRequest;
import com.shop.model.CartItem;
import com.shop.service.CartService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/cart")
@RequiredArgsConstructor
public class CartController {
    private final CartService cart;

    @GetMapping
    public List<CartItem> get(Authentication a) { return cart.get(a.getName()); }

    @PostMapping
    public CartItem add(Authentication a, @Valid @RequestBody CartRequest r) {
        return cart.add(a.getName(), r.productId(), r.quantity());
    }

    @PutMapping("/{itemId}")
    public CartItem update(Authentication a, @PathVariable Long itemId, @RequestParam int quantity) {
        return cart.update(a.getName(), itemId, quantity);
    }

    @DeleteMapping("/{itemId}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void remove(Authentication a, @PathVariable Long itemId) { cart.remove(a.getName(), itemId); }
}
```

src/main/java/com/shop/controller/OrderController.java

```
package com.shop.controller;

import com.shop.model.Order;
import com.shop.model.OrderStatus;
import com.shop.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orders;

    @PostMapping("/api/orders")
    @ResponseStatus(HttpStatus.CREATED)
    public Order checkout(Authentication a) { return orders.checkout(a.getName()); }

    @GetMapping("/api/orders")
    public List<Order> mine(Authentication a) { return orders.myOrders(a.getName()); }

    @GetMapping("/api/orders/{id}")
    public Order one(Authentication a, @PathVariable Long id) { return orders.myOrder(a.getName(), id); }

    @PostMapping("/api/orders/{id}/pay")
    public Order pay(Authentication a, @PathVariable Long id) { return orders.pay(a.getName(), id); }

    @PostMapping("/api/orders/{id}/cancel")
    public Order cancel(Authentication a, @PathVariable Long id) { return orders.cancel(a.getName(), id); }
    // ---- Admin ----
```

```
    @GetMapping("/api/admin/orders")
    public List<Order> all() { return orders.allOrders(); }

    @PutMapping("/api/admin/orders/{id}/status")
    public Order status(@PathVariable Long id, @RequestParam OrderStatus status) {
        return orders.updateStatus(id, status);
    }
}
```

src/main/java/com/shop/controller/ProductController.java

```
package com.shop.controller;

import com.shop.exception.ApiException;
import com.shop.model.Product;
import com.shop.repository.ProductRepository;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductRepository repo;

    @GetMapping
    public Page<Product> list(@RequestParam(defaultValue = "") String q, Pageable pageable) {
        return repo.findByNameContainingIgnoreCase(q, pageable);
    }

    @GetMapping("/{id}")
    public Product get(@PathVariable Long id) {
        return repo.findById(id).orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND, "Product not found"));
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Product create(@Valid @RequestBody Product p) {
        p.setId(null);
        return repo.save(p);
    }

    @PutMapping("/{id}")
    public Product update(@PathVariable Long id, @Valid @RequestBody Product p) {
        get(id);
        p.setId(id);
        return repo.save(p);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        get(id);
        repo.deleteById(id);
    }
}
```

src/main/java/com/shop/dto/Dtos.java

```
package com.shop.dto;

import jakarta.validation.constraints.*;

public class Dtos {
    public record RegisterRequest(@NotBlank String name, @NotBlank @Email String email, @NotBlank @Size(min = 6)
        String password) {}
    public record LoginRequest(@NotBlank String email, @NotBlank String password) {}
    public record AuthResponse(String token, String email, String role) {}
    public record CartRequest(@NotNull Long productId, @Min(1) int quantity) {}
}
```

src/main/java/com/shop/exception/ApiException.java

```
package com.shop.exception;

import org.springframework.http.HttpStatus;
```

```
public class ApiException extends RuntimeException {
    private final HttpStatus status;
    public ApiException(HttpStatus status, String message) {
        super(message);
        this.status = status;
    }
    public HttpStatus getStatus() { return status; }
}
```

src/main/java/com/shop/exception/GlobalExceptionHandler.java

```
package com.shop.exception;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.util.Map;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ApiException.class)
    public ResponseEntity<Map<String, String>> api(ApiException e) {
        return ResponseEntity.status(e.getStatus()).body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> validation(MethodArgumentNotValidException e) {
        Map<String, String> errors = e.getBindingResult().getFieldErrors().stream()
                .collect(Collectors.toMap(f -> f.getField(), f -> String.valueOf(f.getDefaultMessage()), (a, b) ->
                    a));
        return ResponseEntity.badRequest().body(errors);
    }
}
```

src/main/java/com/shop/model/CartItem.java

```
package com.shop.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

@Entity
@Getter @Setter
public class CartItem {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @JsonIgnore
    @ManyToOne(optional = false)
    private User user;
    @ManyToOne(optional = false)
    private Product product;
    private int quantity;
}
```

src/main/java/com/shop/model/Order.java

```
package com.shop.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "orders")
@Getter @Setter
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @JsonIgnore
    @ManyToOne(optional = false)
```

```
    private User user;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    private List<OrderItem> items = new ArrayList<>();
    private BigDecimal total;
    @Enumerated(EnumType.STRING)
    private OrderStatus status = OrderStatus.PLACED;
    private LocalDateTime createdAt;

    @PrePersist
    void onCreate() { createdAt = LocalDateTime.now(); }
}
```

src/main/java/com/shop/model/OrderItem.java

```
package com.shop.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import java.math.BigDecimal;

@Entity
@Getter @Setter
public class OrderItem {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @JsonIgnore
    @ManyToOne(optional = false)
    private Order order;
    @ManyToOne(optional = false)
    private Product product;
    private int quantity;
    private BigDecimal price; // purchase-time price snapshot
}
```

src/main/java/com/shop/model/OrderStatus.java

```
package com.shop.model;
public enum OrderStatus { PLACED, PAID, SHIPPED, DELIVERED, CANCELLED }
```

src/main/java/com/shop/model/Product.java

```
package com.shop.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.Getter;
import lombok.Setter;
import java.math.BigDecimal;

@Entity
@Getter @Setter
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank
    private String name;
    @Column(length = 1000)
    private String description;
    @NotNull @DecimalMin("0.01")
    private BigDecimal price;
    @Min(0)
    private int stock;
    private String category;
}
```

src/main/java/com/shop/model/Role.java

```
package com.shop.model;
public enum Role { USER, ADMIN }
```

src/main/java/com/shop/model/User.java

```
package com.shop.model;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
@Entity
```

```
@Table(name = "users")
@Getter @Setter
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    @Column(unique = true, nullable = false)
    private String email;
    @Column(nullable = false)
    private String password;
    @Enumerated(EnumType.STRING)
    private Role role = Role.USER;
}
```

src/main/java/com/shop/repository/CartItemRepository.java

```
package com.shop.repository;

import com.shop.model.CartItem;
import com.shop.model.Product;
import com.shop.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;

public interface CartItemRepository extends JpaRepository<CartItem, Long> {
    List<CartItem> findByUser(User user);
    Optional<CartItem> findByUserAndProduct(User user, Product product);
}
```

src/main/java/com/shop/repository/OrderRepository.java

```
package com.shop.repository;

import com.shop.model.Order;
import com.shop.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByUserOrderByCreatedAtDesc(User user);
}
```

src/main/java/com/shop/repository/ProductRepository.java

```
package com.shop.repository;

import com.shop.model.Product;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {
    Page<Product> findByNameContainingIgnoreCase(String q, Pageable pageable);
}
```

src/main/java/com/shop/repository/UserRepository.java

```
package com.shop.repository;

import com.shop.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

src/main/java/com/shop/security/JwtFilter.java

```
package com.shop.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
```

```
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;
import java.util.List;

@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {
    private final JwtService jwt;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                Claims c = jwt.parse(header.substring(7));
                var auth = new UsernamePasswordAuthenticationToken(
                        c.getSubject(), null,
                        List.of(new SimpleGrantedAuthority("ROLE_" + c.get("role", String.class))));
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException | IllegalArgumentException ignored) {
                // invalid/expired token -> request stays unauthenticated
            }
        }
        chain.doFilter(req, res);
    }
}
```

src/main/java/com/shop/security/JwtService.java

```
package com.shop.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Service
public class JwtService {
    private final SecretKey key;
    private final long expirationMs;

    public JwtService(@Value("${app.jwt.secret}") String secret,
                      @Value("${app.jwt.expiration-ms}") long expirationMs) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationMs = expirationMs;
    }

    public String generate(String email, String role) {
        Date now = new Date();
        return Jwts.builder()
                .subject(email)
                .claim("role", role)
                .issuedAt(now)
                .expiration(new Date(now.getTime() + expirationMs))
                .signWith(key)
                .compact();
    }

    public Claims parse(String token) {
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload();
    }
}
```

src/main/java/com/shop/security/SecurityConfig.java

```
package com.shop.security;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtFilter jwtFilter;

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(c -> c.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(e -> e.authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
                .requestMatchers("/api/products/**", "/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

src/main/java/com/shop/service/CartService.java

```
package com.shop.service;

import com.shop.exception.ApiException;
import com.shop.model.*;
import com.shop.repository.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service
@RequiredArgsConstructor
public class CartService {
    private final CartItemRepository cartRepo;
    private final ProductRepository productRepo;
    private final UserRepository userRepo;

    private User user(String email) {
        return userRepo.findByEmail(email)
                .orElseThrow(() -> new ApiException(HttpStatus.UNAUTHORIZED, "User not found"));
    }

    public List<CartItem> get(String email) {
        return cartRepo.findByUser(user(email));
    }

    @Transactional
    public CartItem add(String email, Long productId, int qty) {
        User u = user(email);
        Product p = productRepo.findById(productId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND, "Product not found"));
        CartItem item = cartRepo.findByUserAndProduct(u, p).orElseGet(() -> {
            CartItem c = new CartItem();
            c.setUser(u);
            c.setProduct(p);
            return c;
        });
        int newQty = item.getQuantity() + qty;
        if (newQty > p.getStock()) throw new ApiException(HttpStatus.BAD_REQUEST, "Not enough stock");
        item.setQuantity(newQty);
        return cartRepo.save(item);
    }

    @Transactional
    public CartItem update(String email, Long itemId, int qty) {
```

```
        CartItem item = own(email, itemId);
        if (qty < 1) throw new ApiException(HttpStatus.BAD_REQUEST, "Quantity must be >= 1");
        if (qty > item.getProduct().getStock()) throw new ApiException(HttpStatus.BAD_REQUEST, "Not enough stock");
        item.setQuantity(qty);
        return cartRepo.save(item);
    }

    @Transactional
    public void remove(String email, Long itemId) {
        cartRepo.delete(own(email, itemId));
    }

    private CartItem own(String email, Long itemId) {
        return cartRepo.findById(itemId)
                .filter(c -> c.getUser().getEmail().equals(email))
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND, "Cart item not found"));
    }
}
```

src/main/java/com/shop/service/OrderService.java

```
package com.shop.service;

import com.shop.exception.ApiException;
import com.shop.model.*;
import com.shop.repository.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.math.BigDecimal;
import java.util.List;

@Service
@RequiredArgsConstructor
public class OrderService {
    private final OrderRepository orderRepo;
    private final CartItemRepository cartRepo;
    private final UserRepository userRepo;

    private User user(String email) {
        return userRepo.findByEmail(email)
                .orElseThrow(() -> new ApiException(HttpStatus.UNAUTHORIZED, "User not found"));
    }

    @Transactional
    public Order checkout(String email) {
        User u = user(email);
        List<CartItem> cart = cartRepo.findByUser(u);
        if (cart.isEmpty()) throw new ApiException(HttpStatus.BAD_REQUEST, "Cart is empty");

        Order order = new Order();
        order.setUser(u);
        BigDecimal total = BigDecimal.ZERO;
        for (CartItem c : cart) {
            Product p = c.getProduct();
            if (c.getQuantity() > p.getStock())
                throw new ApiException(HttpStatus.BAD_REQUEST, "Not enough stock for " + p.getName());
            p.setStock(p.getStock() - c.getQuantity());
            OrderItem oi = new OrderItem();
            oi.setOrder(order);
            oi.setProduct(p);
            oi.setQuantity(c.getQuantity());
            oi.setPrice(p.getPrice());
            order.getItems().add(oi);
            total = total.add(p.getPrice().multiply(BigDecimal.valueOf(c.getQuantity())));
        }
        order.setTotal(total);
        Order saved = orderRepo.save(order);
        cartRepo.deleteAll(cart);
        return saved;
    }

    public List<Order> myOrders(String email) {
        return orderRepo.findByUserOrderByCreatedAtDesc(user(email));
    }

    public Order myOrder(String email, Long id) {
        return orderRepo.findById(id)
                .filter(o -> o.getUser().getEmail().equals(email))
```

```
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND, "Order not found"));
    }

    /** Mock payment: sirf status PAID karta hai. Real gateway (Razorpay/Stripe) yahan integrate karo. */
    @Transactional
    public Order pay(String email, Long id) {
        Order o = myOrder(email, id);
        if (o.getStatus() != OrderStatus.PLACED)
            throw new ApiException(HttpStatus.BAD_REQUEST, "Order cannot be paid in status " + o.getStatus());
        o.setStatus(OrderStatus.PAID);
        return orderRepo.save(o);
    }

    @Transactional
    public Order cancel(String email, Long id) {
        Order o = myOrder(email, id);
        if (o.getStatus() != OrderStatus.PLACED)
            throw new ApiException(HttpStatus.BAD_REQUEST, "Only PLACED orders can be cancelled");
        o.getItems().forEach(i -> i.getProduct().setStock(i.getProduct().getStock() + i.getQuantity()));
        o.setStatus(OrderStatus.CANCELLED);
        return orderRepo.save(o);
    }

    public List<Order> allOrders() { return orderRepo.findAll(); }

    @Transactional
    public Order updateStatus(Long id, OrderStatus status) {
        Order o = orderRepo.findById(id)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND, "Order not found"));
        o.setStatus(status);
        return orderRepo.save(o);
    }
}
```